

# PROGETTO CITY PLANNER

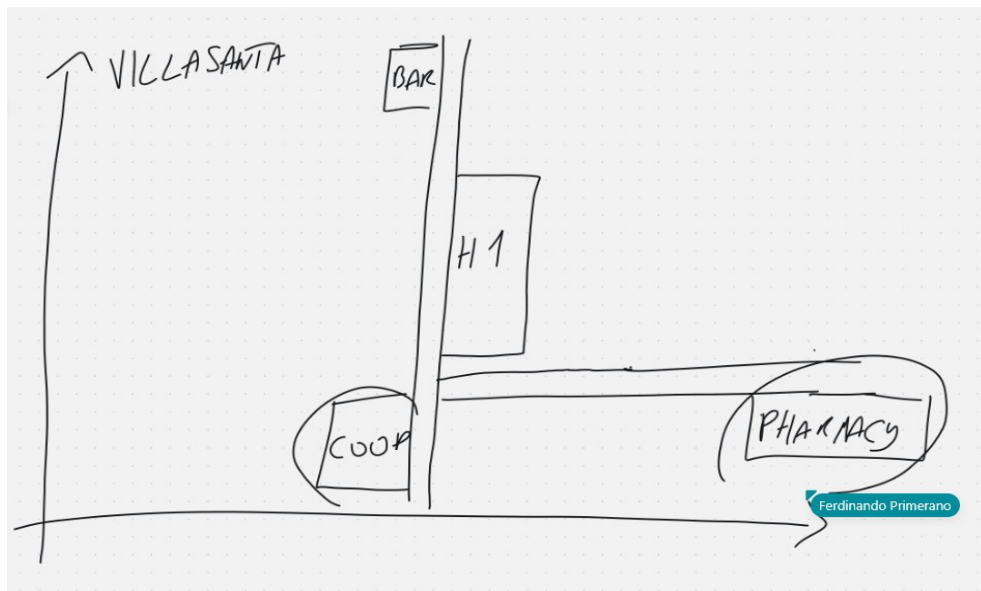
*Analisi, Codice Fornito dal Professore e Richieste*

---

## 1. Descrizione del Progetto

Il progetto City Planner calcola il valore di una casa in base ai servizi disponibili nelle sue vicinanze. Il valore dipende sia dalle dimensioni dell'abitazione che dalla prossimità a servizi come negozi, ristoranti, stazioni di trasporto e altro ancora.

**Obiettivo principale:** calcolare le distanze tra una casa e tutti i servizi presenti nella città, inclusa la stazione.



Schema generale: ogni servizio offre qualcosa e contribuisce al valore complessivo di un immobile.

---

## 2. Setup del Progetto Angular

Creare un progetto Angular chiamato CP:

```
ng new CP
```

### 2.1 Gestione Branch (Git)

Creare il branch di sviluppo e spostarsi su di esso:

```
git checkout -b dev-house
```

⚠ **Importante:** verificare sempre di essere sulla branch corretta prima di effettuare modifiche.

```
src > model > TS entities.ts > ...  
1  export interface House{  
2      id?:number;  
3      x:number;  
4      y:number;  
5      address:string;  
6      capacity:number;  
7  }  
8
```

## 3. Modello dei Dati

### 3.1 ServiceType

Definire un tipo ServiceType per categorizzare i servizi disponibili in città (es. FOOD, TRANSPORT, ecc.).

```
export interface Asset{  
    id?:number;  
    assetType:'FOOD' | 'MEDICINE' | 'CLINIC' | 'SECURITY' | 'EDUCATION' | 'FUN' | 'TRANSPORT';  
    x:number;  
    y:number;  
}
```

## 3.2 Classe City

La città è rappresentata come una classe. L'origine è posizionata in alto a sinistra e la città è modellata come un quadrato. Caricare i dati dal backend sarà affrontato in una fase successiva.

```
export class City{
  id:number;
  name:string;
  houses:House[];
  assets:Asset[];

  constructor(id:number,name:string,houses:House[],assets:Asset[]) {
    this.id = id;
    this.name = name;
    this.houses = houses;
    this.assets = assets;
  }
}

getClosestAsset(x:number,y:number, type:string):Asset | null{
  let closestAsset:Asset | null = null;
  let minDistance = Infinity;
  this.assets.forEach(asset => {
    // manhattan norm
    const distance = Math.abs(asset.x - x) + Math.abs(asset.y - y);
    if (distance < minDistance && asset.assetType==type) {
      minDistance = distance;
      closestAsset = asset;
    }
  });
  return closestAsset;
}
```

## 4. Codice Fornito dal Professore

| ■ Metodo <code>getClosestAsset</code> — Distanza di Manhattan                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Il metodo riceve tre parametri: le coordinate x e y di un punto, e una stringa type che indica il tipo di asset cercato.                                 |
| Vengono inizializzate due variabili: <code>closestAsset = null</code> e <code>minDistance = Infinity</code> .                                            |
| Si scorrono tutti gli asset in <code>this.assets</code> con un <code>forEach</code> .                                                                    |
| Per ciascun asset si calcola la distanza di Manhattan: somma delle differenze assolute tra le coordinate.                                                |
| Se l'asset è del tipo corretto e la distanza è minore di <code>minDistance</code> , si aggiornano <code>closestAsset</code> e <code>minDistance</code> . |
| Al termine, si restituisce l'asset più vicino oppure null se non trovato.                                                                                |

**Nota sulla Distanza di Manhattan:** non sfrutta la diagonale — è la somma delle differenze assolute sugli assi X e Y.

### Formula

```
distanza = |x_asset - x_punto| + |y_asset - y_punto|
```

### Nota su `forEach` e Arrow Function

**`forEach`** prende un array e applica la funzione passata come argomento a ogni elemento dell'array, uno per uno.

Quando il professore parla di "array function" intende **una funzione che viene passata come argomento a un metodo di un array** e che viene eseguita automaticamente per ciascun elemento. La arrow function `=>` è la sintassi con cui quella funzione viene scritta.

---

## 5. Mock City Service

### 5.1 Cosa è un Service Angular

Un servizio in Angular è una classe che contiene logica e dati condivisibili tra più componenti.

#### ■ Perché si usa un Service?

I componenti dovrebbero occuparsi solo della visualizzazione.

Logica di business, chiamate HTTP e gestione dei dati vanno nei servizi.

Il codice risulta organizzato e riutilizzabile.

Uno degli usi più comuni è comunicare con le API tramite il modulo HttpClient.

### 5.2 Implementazione del Mock

```
> app > TS mock-city-service.ts > MockCityService > getCity
4
5 @Injectable({
6   providedIn: 'root',
7 })
8 export class MockCityService {
9
10   city: City = new City
11   (
12     1,
13     "Villasanta",
14     [{id:1, x:5, y:5, address:'Via dei Tigli 10', capacity:200}],
15     [
16       {id:1, x:4, y:5, name:'Bar Sport', address:'Via dei Tigli 1', assetType:'FOOD'},
17       {id:2, x:4, y:3, name:'Bar Carino', address:'Via dei Gatti 1', assetType:'FOOD'},
18     ]
19   );
20
21   getCity(id:number):Observable<City>{
22     return of(this.city)
23   }
24 }
25
26
27
```

Il mock riceve un `id` ma lo ignora completamente, restituendo sempre la stessa città. `of()` è un operatore di RxJS che trasforma un valore normale in un Observable, simulando una vera chiamata HTTP.

#### ■ A cosa serve un Mock Service?

Testare il frontend senza dipendere dal server.

Avere dati fissi e prevedibili per le prove.

Sostituirlo facilmente con il servizio reale quando il backend è pronto.

```
export class City{
  id:number;
  name:string;
  houses:House[];
  assets:Asset[];

  constructor(id:number,name:string,houses:House[],assets:Asset[]) {
    this.id = id;
    this.name = name;
    this.houses = houses;
    this.assets = assets
  }
}
```

## 6. Test con Jasmine

### 6.1 File spec.ts

Ogni componente e servizio Angular nasce con un file spec.ts. Per eseguire tutti i test:

```
ng test
```

```
describe('MockCityService', () => {

  let service: MockCityService;

  beforeEach(() => {
    TestBed.configureTestingModule({});
    service = TestBed.inject(MockCityService);
  });

  it('should be created', () => {
    expect(service).toBeTruthy();
  });

  it('should return Villasanta', () => {
    service.getCity(1).subscribe(city => {
      expect(city.name).toBe('Villasanta');
    });
  });
});
```

### 6.2 Come funziona Jasmine

#### ■ Struttura dei Test Jasmine (Codice Professore)

describe — raggruppa tutti i test di una classe o funzione.

it — definisce il singolo test con un'etichetta descrittiva.

expect — fa l'asserzione, confrontando il risultato ottenuto con quello atteso.

Confronto con Java:

| Java (JUnit)                                | Angular (Jasmine)                          |
|---------------------------------------------|--------------------------------------------|
| <code>assertEquals(expected, actual)</code> | <code>expect(actual).toBe(expected)</code> |

## 6.3 Test del MockCityService

```
});  
  
// sto gestando il metodo getClosestAsset  
it('closest to home is bar sport', ()=>{  
  service.getCity(1).subscribe(city => {  
    const closestAsset = city.getClosestAsset(5,5,'FOOD');  
    expect(closestAsset?.name).toBe('Bar Sport');  
  });  
});  
});
```

### ■ Spiegazione del Test (Codice Professore)

Chiama `getCity(1)` sul servizio.

Poiché restituisce un `Observable<City>`, si fa `.subscribe()` per ottenere il valore.

Una volta ricevuta la città, verifica che il name sia esattamente 'Villasanta'.

Jasmine usa `expect(...).toBe(...)` che si legge come inglese naturale: "mi aspetto che `city.name` sia Villasanta".

## 6.4 Test su getClosestAsset

```
});

// sto gestando il metodo getClosestAsset
it('closest to home is bar sport', ()=>{
  service.getCity(1).subscribe(city => {
    const closestAsset = city.getClosestAsset(5,5, 'FOOD');
    expect(closestAsset?.name).toBe('Bar Sport');
  });
});
});
```

Passi del test:

1. **Carica la città:** `service.getCity(1).subscribe(city => ...)`
2. **Chiama il metodo:** `city.getClosestAsset(5, 5, 'FOOD')` — cerca l'asset FOOD più vicino a (5,5).
3. **Verifica:** `expect(closestAsset?.name).toBe('Bar Sport')`

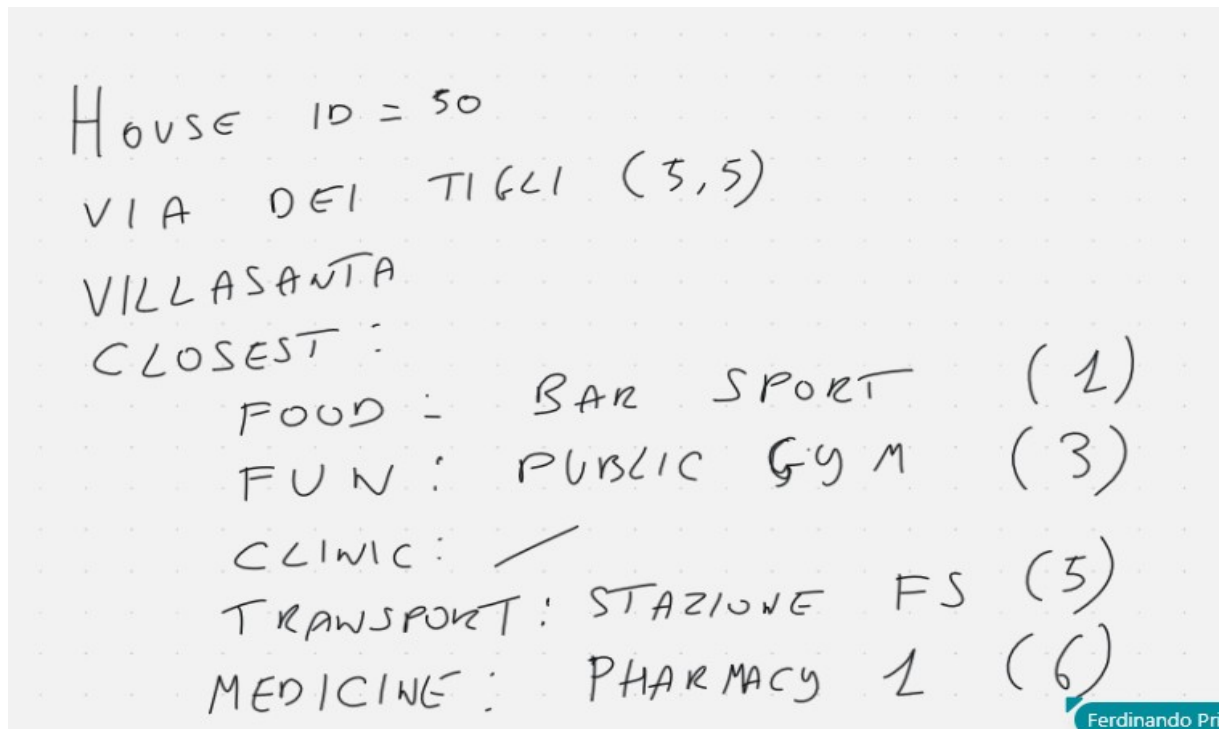
Perché `?.` (optional chaining)? Poiché `getClosestAsset` può restituire null, il `?.` evita un errore: se è null, restituisce undefined invece di crashare.

## 7. Task Richieste

### TASK — Branch: dev-house-component

- Implementare il componente `HouseComponent`.
- Il componente deve trovare tutti i servizi più vicini fino a un certo massimo.
- Scrivere anche il test del componente (file `spec.ts`).





Screenshot del risultato atteso / codice di riferimento per il componente.

## 8. Riepilogo Concetti Chiave

| Concetto                      | Descrizione                                                                                                   |
|-------------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Distanza di Manhattan</b>  | Somma delle differenze assolute: $ \Delta x  +  \Delta y $ . Non usa la diagonale.                            |
| <b>Service Angular</b>        | Classe con logica e dati condivisibili tra componenti. Usato per HTTP, business logic, etc.                   |
| <b>Observable / RxJS</b>      | Struttura per gestire dati asincroni. Si usa <code>.subscribe()</code> per ricevere il valore.                |
| <b>of()</b>                   | Operatore RxJS che converte un valore sincrono in Observable (usato nei mock).                                |
| <b>Mock Service</b>           | Servizio finto con dati fissi, usato durante lo sviluppo quando il backend non è pronto.                      |
| <b>Jasmine / spec.ts</b>      | Framework di test Angular. <code>describe</code> / <code>it</code> / <code>expect</code> sostituiscono JUnit. |
| <b>Optional Chaining (?.)</b> | Evita crash se il valore è null: restituisce undefined invece di errore.                                      |
| <b>Arrow Function (=&gt;)</b> | Sintassi compatta per funzioni passate come argomento (es. in <code>forEach</code> ).                         |